

## Build & Flash Firmware qua Command Line Rời IDE – Làm chủ Toolchain Nhúng

### Buổi 03: CMake, Ninja & STM32\_Programmer\_CLI

Board thực hành: **STM32 Blue Pill (STM32F103C8T6)** – mạch nạp ST-Link/V2

---

Machine Learning & IoT Lab (ML IoT)  
Ho Chi Minh City University of Technology (HCMUT)

1. Định hướng & kiểm tra môi trường
2. Luồng biên dịch: từ .c tới .bin
3. Bộ nhớ MCU & Linker script
4. CMake – mô tả dự án một lần
5. Ninja & tự động hóa bằng script
6. Nạp firmware qua SWD & CLI
7. Tổng kết & hướng tới CI/CD

## Cần chuẩn bị (tại nhà, trước buổi học)

**Phần cứng:** Board STM32 Blue Pill + mạch nạp ST-Link/V2

**Dự án thực hành:**  
`i2c_demo_baremetal`

**Công cụ cần cài:**  
arm-none-eabi-gcc, CMake, Ninja,  
STM32CubeProgrammer

Hướng dẫn cài đặt chi tiết từng bước cho Windows & Linux – ở các slide kế tiếp.

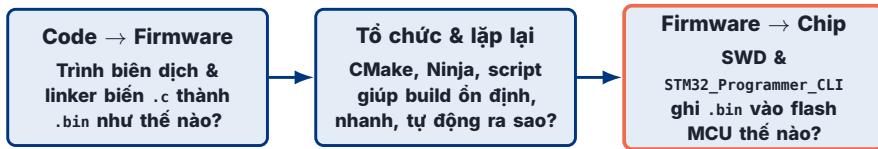
## Giới hạn của IDE “bấm nút”

- ▶ Bấm *Build, Flash* nhưng **không thấy** lệnh nào thật sự chạy bên dưới.
- ▶ Mỗi IDE (Keil, IAR, STM32CubeIDE) cấu hình theo cách riêng, khó tái lập trên máy khác.
- ▶ Không chạy được trên máy chủ **không màn hình** (build server, CI/CD).
- ▶ Khi lỗi, thông báo bị IDE “gói lại”, khó lần ra nguyên nhân gốc.

## Mục tiêu buổi học

- ▶ Hiểu **bản chất** chuỗi biên dịch → nạp firmware.
- ▶ Làm chủ **dòng lệnh**: `arm-none-eabi-gcc`, `CMake`, `Ninja`, `STM32_Programmer_CLI`.
- ▶ Gói toàn bộ vào **một script** tái sử dụng → nền tảng cho CI/CD.

IDE không sai – nó nhanh và tiện. Nhưng nó *giấu* toolchain đi. Khi bạn tự gõ được từng lệnh, bạn có thể tái lập chính xác quy trình đó ở bất cứ đâu: máy đồng nghiệp, container Docker, hay một build server tự động.



- ▶ Ba khối này ghép lại thành **một dòng chảy duy nhất**: sửa code → board chạy chương trình mới.
- ▶ Mọi ví dụ trong buổi học lấy trực tiếp từ dự án thật `i2c_demo_baremetal` (đọc cảm biến AHT20 qua I2C, log qua UART).

# HƯỚNG DẪN CÀI ĐẶT

**Làm ở nhà – từng bước một**

Dành cho người mới: không bỏ sót bước nào

# Cài đặt: ta cần bốn công cụ – chỉ một lần duy nhất



ML IoT  
HCMUT EE MACHINE LEARNING & IOT LAB

Bốn công cụ dưới đây ghép thành “dây chuyền” biến code thành firmware chạy trên board. Cài một lần ở nhà, dùng cho cả khóa học.

| Công cụ             | Vai trò (nói đơn giản)                | Lấy ở đâu    |
|---------------------|---------------------------------------|--------------|
| arm-none-eabi-gcc   | dịch code C thành mã máy cho chip ARM | winget / apt |
| cmake               | mô tả dự án, sinh cấu hình build      | winget / apt |
| ninja               | chạy các lệnh biên dịch (rất nhanh)   | winget / apt |
| STM32CubeProgrammer | nạp firmware xuống board qua ST-Link  | trang chủ ST |

## Chuẩn bị trước

Máy có Internet, còn khoảng **2 GB** trống, có **quyền Admin** (Windows) hoặc **sudo** (Linux). Dành ra chừng 30 phút.

## Đọc bản đồ các slide

Ba công cụ đầu cài **tự động**. Riêng CubeProgrammer phải tải tay (hay vướng nhất). Dùng **Windows** thì theo các slide có chữ *Windows*; dùng **Linux** thì theo các slide chữ *Linux*.

# Bước 0 – Mở “cửa sổ dòng lệnh” (Terminal)

Suốt buổi học ta **gõ lệnh** vào một cửa sổ Terminal thay vì bấm nút. Mở nó lên trước đã:

## Trên Windows

- ▶ Nhấn phím **Windows**, gõ PowerShell, mở **Windows PowerShell**.
- ▶ Hoặc: chuột phải nút **Start** → *Terminal* / *Windows PowerShell*.

## Trên Linux (Ubuntu)

- ▶ Nhấn **Ctrl + Alt + T**.
- ▶ Hoặc tìm *Terminal* trong danh sách ứng dụng.

Cửa sổ hiện **con trỏ nhấp nháy** nghĩa là nó đang chờ bạn gõ. Gõ (hoặc **dán**) một dòng lệnh rồi nhấn **Enter** để chạy. Mẹo: nên **copy lệnh** từ tài liệu rồi dán (trong PowerShell bấm chuột phải để dán) để khỏi gõ sai từng ký tự.

winget là “chợ ứng dụng dòng lệnh” có sẵn trên Windows 10/11 – nó **tự tải và cài** phần mềm giúp bạn. Mở PowerShell, gõ:

```
winget --version
```

## Thành công khi...

In ra một số phiên bản, ví dụ v1.8.xxxx. Vậy là dùng được, sang slide kế.

## Nếu báo “not recognized”

Windows quá cũ hoặc thiếu **App Installer**. Mở **Microsoft Store** → tìm App Installer → bấm *Update/Get*. Xong **mở PowerShell mới** thử lại.





Dán **từng dòng**, nhấn Enter, chờ dòng này chạy xong rồi mới sang dòng kế:

```
winget install --id Arm.GnuArmEmbeddedToolchain  
winget install --id Kitware.CMake  
winget install --id Ninja-build.Ninja
```

## Bạn sẽ thấy gì

- ▶ Thanh tải phần trăm, rồi dòng Successfully installed.
- ▶ Nếu hỏi đồng ý điều khoản: gõ Y rồi Enter.

## Trục trặc nhỏ

- ▶ Hỏi chọn nguồn msstore/winget → chọn winget.
- ▶ Báo already installed → tốt, bỏ qua.

Cài xong cả ba: **đóng PowerShell rồi mở cửa sổ mới** để PATH kịp cập nhật (nếu không, lệnh sẽ "not recognized" dù đã cài).

Công cụ này **không có** trên winget, phải tải tay từ trang chủ ST (miễn phí):

1. Lên trang chủ **ST**, tìm STM32CubeProgrammer, tải bản **Windows** (file .zip).
2. Tạo **tài khoản ST miễn phí** (bắt buộc mới tải được) – link tải gửi qua email.
3. Giải nén .zip, chạy SetupSTM32CubeProgrammer-x.x.x.exe, bấm **Next** liên tục, **giữ nguyên** thư mục cài mặc định.
4. Khi hỏi cài **driver ST-Link** → đồng ý (*Install*). Driver này giúp Windows nhận mạch nạp.

**Ghi nhớ đường dẫn cài** – slide sau cần nó. Mặc định thường là:

```
C:\Program Files\STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin
```

winget tự thêm PATH cho 3 công cụ đầu; CubeProgrammer thì **không**, nên Windows chưa biết STM32\_Programmer\_CLI nằm đâu. Ta “chỉ đường” cho nó:

1. Copy đường dẫn thư mục ...\\bin (ở slide trước).
2. Nhấn **Windows**, gõ `environment variables` → mở *Edit the system environment variables*.
3. Bấm **Environment Variables...** → ở ô *User variables* chọn dòng `Path` → **Edit...**
4. Bấm **New**, dán đường dẫn bin vào, bấm **OK** cả ba cửa sổ.
5. **Đóng và mở PowerShell mới**, rồi kiểm tra:

```
STM32_Programmer_CLI --version
```

In ra phiên bản = xong. Vẫn “not recognized”? Đường dẫn dán sai (phải trỏ đúng thư mục chứa file .exe, kết thúc bằng \\bin) hoặc bạn chưa mở terminal mới.

| Thông báo lỗi                         | Nguyên nhân                           | Cách sửa                                            |
|---------------------------------------|---------------------------------------|-----------------------------------------------------|
| 'winget' ... not recognized           | thiếu App Installer / Windows cũ      | cài App Installer từ Store, mở terminal mới         |
| 'cmake'/'ninja' not recognized        | chưa mở terminal mới sau khi cài      | đóng & mở lại PowerShell                            |
| 'STM32_Programmer_CLI' not recognized | chưa thêm PATH / chưa mở terminal mới | làm lại slide 4/4, mở cửa sổ mới                    |
| running scripts is disabled           | chính sách chạy script bị chặn        | Set-ExecutionPolicy -Scope CurrentUser RemoteSigned |
| winget tải chậm / đứng                | mạng yếu hoặc server bận              | chờ, hoặc chạy lại lệnh                             |

Quy tắc nhớ: gần như mọi lỗi "not recognized" đều sửa được bằng **cài lại đúng + mở cửa sổ terminal mới**.

Áp dụng cho Ubuntu/Debian. apt là trình quản lý gói của hệ thống – gõ hai lệnh:

```
sudo apt update  
sudo apt install -y gcc-arm-none-eabi cmake ninja-build
```

## Giải thích cho người mới

- ▶ sudo = chạy với quyền quản trị; máy hỏi **mật khẩu đăng nhập** (gõ không hiện ký tự – cứ gõ rồi Enter).
- ▶ apt update làm mới danh sách gói; apt install tải & cài.

## Distro khác?

Fedora dùng dnf, Arch dùng pacman (đổi tên gói tương ứng). Báo Unable to locate package → chạy sudo apt update trước & kiểm tra chính tả.

CubeProgrammer **không có** trên apt. Tải file .zip từ trang chủ ST (cần tài khoản miễn phí), rồi trong Terminal:

```
sudo apt install -y default-jre unzip # bo cai la chuong trinh Java
unzip en.stm32cubeprg-lin-*.zip
./SetupSTM32CubeProgrammer-*.linux
```

## Chạy bộ cài

Bộ cài mở cửa sổ đồ họa (Java) → bấm **Next** như trên Windows. Nếu báo Permission denied khi chạy file, cấp quyền chạy: `chmod +x Setup*.linux`.

## Thiếu Java?

Chạy bộ cài báo no java / command not found → bạn chưa cài default-jre. Cài dòng đầu tiên ở trên rồi thử lại.

Chỉ cho shell biết CubeProgrammer ở đâu – thêm 2 dòng vào **cuối** ~/.bashrc:

```
export STM32_PRG_PATH=$HOME/STMicroelectronics/STM32Cube/STM32CubeProgrammer/bin
export PATH="$STM32_PRG_PATH:$PATH"
```

Nạp lại cấu hình: `source ~/.bashrc` (hoặc mở terminal mới).

**Bước riêng của Linux – udev rules:** cắm ST-Link, lsusb thấy 0483:3748 nhưng khi flash vẫn báo Permission denied (errno=13) vì user thường chưa có quyền ghi device. Cài rule **một lần**:

```
sudo cp "$STM32_PRG_PATH/../../Drivers/rules/*.rules /etc/udev/rules.d/
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Sau khi cài rule, **rút và cắm lại** ST-Link để quyền mới có hiệu lực.

| Thông báo lỗi                                                                | Nguyên nhân                                | Cách sửa                                                                             |
|------------------------------------------------------------------------------|--------------------------------------------|--------------------------------------------------------------------------------------|
| command not found<br>(gcc/cmake/ninja)                                       | chưa cài hoặc gõ sai tên                   | apt install lại, kiểm tra chính tả                                                   |
| STM32_Programmer_CLI: not found<br>Permission denied (errno=13) khi<br>flash | PATH sai / chưa nạp lại<br>thiếu udev rule | sửa ~/.bashrc, chạy source ~/.bashrc<br>cài udev rule (slide trước), cắm lại ST-Link |
| bộ cài không chạy / no java<br>Unable to locate package                      | thiếu Java (JRE)<br>danh sách gói cũ       | sudo apt install default-jre<br>sudo apt update rồi cài lại                          |





```
arm-none-eabi-gcc --version    # trình biên dịch chéo (cross-compiler) cho ARM
cmake --version               # sinh cấu hình build
ninja --version                # công cụ build thực thi
STM32_Programmer_CLI --version # nạp firmware qua ST-Link/SWD
```

| Công cụ              | Vai trò trong buổi học                       |
|----------------------|----------------------------------------------|
| arm-none-eabi-gcc    | dịch .c/.s thành mã máy Cortex-M3            |
| cmake                | đọc CMakeLists.txt → sinh build.ninja        |
| ninja                | chạy các lệnh biên dịch, song song, tăng dần |
| STM32_Programmer_CLI | ghi .bin vào flash 0x08000000 rồi reset      |

Nếu một lệnh báo "command not found" → công cụ chưa nằm trong biến môi trường PATH. Ta sẽ kiểm tra ngay tại lớp.

## Bước 1 – Cấu hình & biên dịch:

```
cmake -B build -G Ninja      # tạo thư mục build/, sinh build.ninja
cmake --build build          # gọi Ninja biên dịch toàn bộ
```

### Thành công khi...

Trong build/ xuất hiện ba file cùng tên:

```
stm32f103_aht20_baremetal.elf
... .hex  ... .bin
```

và dòng Memory region ... % used in ra ở cuối.

### Nếu lỗi?

Giờ tay – ta cùng đọc thông báo lỗi và kiểm tra PATH của toolchain ngay bây giờ.

*Ghi chú:* ba file .elf/.hex/.bin do lệnh objcopy trong CMakeLists.txt tự sinh – ta sẽ mổ xẻ ở Phần 4.

1. **Đọc kỹ dòng lỗi cuối cùng** – nó thường nói thẳng đang thiếu gì.
2. "not recognized" / "command not found" = công cụ **chưa vào PATH**: mở terminal **mới** và kiểm tra lại phần cài đặt của công cụ đó.
3. Sau khi **cài mới** hoặc **đổi PATH**, luôn **mở cửa sổ terminal mới** rồi mới thử.
4. Chạy lại bốn lệnh --version để biết **chính xác** công cụ nào còn thiếu.
5. Vẫn kẹt: copy **nguyên văn** dòng lỗi + ảnh màn hình, hỏi trên nhóm lớp / AI bất kì hoặc mang máy đến sớm 10 phút.

**90% lỗi cài đặt của người mới** rơi vào ba nguyên nhân: (a) chưa mở terminal mới, (b) PATH thiếu hoặc sai, (c) gõ sai tên lệnh. Kiểm tra ba thứ này **đầu tiên**.

# PHẦN 1

**Luồng biên dịch: từ .c tới .bin**

Preprocess → Compile → Assemble → Link



- ▶ **Preprocess:** xử lý `#include`, `#define`, `#ifdef` – dán nội dung header, thay macro. Kết quả là một file C “phẳng”.
- ▶ **Compile:** dịch C sang **assembly** của kiến trúc đích (Cortex-M3).
- ▶ **Assemble:** dịch assembly sang **mã máy**, đóng gói thành **object file** `.o`.
- ▶ **Link:** ghép mọi `.o` + thư viện, gán địa chỉ theo linker script → `.elf` hoàn chỉnh.

Lệnh gcc thường làm cả bốn bước trong một lần gọi – nhưng hiểu tách bạch giúp bạn **đọc đúng tầng gây lỗi**.

Trong `main.c`, các macro cấu hình cảm biến AHT20 được thay thế **trước khi** biên dịch:

```
#define AHT20_ADDRESS 0x38
uint8_t init_cmd[3] = {AHT20_CMD_INIT, 0x08, 0x00};
i2c_write(I2C1, AHT20_ADDRESS, init_cmd, 3); // 0x38 được thay vào đây
```

## Preprocessor làm gì

- ▶ Dán toàn bộ header (`#include "stm32f103xb.h"`) vào file.
- ▶ Thay mọi macro bằng giá trị thật.
- ▶ Chọn nhánh `#ifdef` (VD: `DEBUG`).

## Vì sao cần biết

Lỗi kiểu `"AHT20_ADDRESS undeclared"` hay macro sai giá trị → vấn đề nằm ở **tầng preprocess**, không phải logic. Xem file đã bung bằng `arm-none-eabi-gcc -E`.

Toolchain của dự án bật cảnh báo nghiêm ngặt (trong `gcc-arm-none-eabi.cmake`):

```
-Wall -Wextra -Wpedantic    # bat gan nhu moi canh bao huu ich
-00 -g3                     # Debug: khong toi uu, day du thong tin debug
-0s -g0                     # Release: toi uu dung luong (flash nho)
```

## Đọc thông báo compiler

- ▶ error: → build dừng, phải sửa (VD thiếu ;, sai kiểu).
- ▶ warning: → build vẫn chạy nhưng là **mùi lỗi** (biến chưa dùng, so sánh sai dấu).

## -00 VS -0s

- ▶ -00 -g3: dễ debug từng dòng, code lớn hơn.
- ▶ -0s: nén chặt cho vừa 64 KB flash của Blue Pill.

Cờ `-ffunction-sections` `-fdata-sections` đặt mỗi hàm/biến vào một section riêng – chuẩn bị cho linker **loại bỏ code chết** ở giai đoạn 4.



`.text`  
mã lệnh (hàm)

`.rodata`  
hằng số, chuỗi "..."

`.data`  
biến toàn cục  
có giá trị đầu

`.bss`  
biến toàn cục = 0

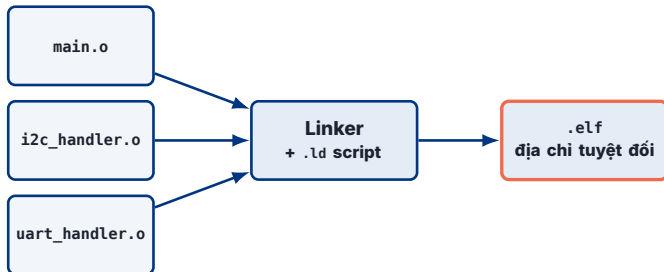
bảng symbol  
+ relocation

## Điểm mấu chốt

- ▶ Mỗi file .c dịch **độc lập** thành một .o – chưa biết địa chỉ tuyệt đối.
- ▶ Hàm gọi tới hàm ở file khác (VD `main.c` gọi `i2c_write`) tạm để **ô trống** + một ghi chú "relocation".
- ▶ Bảng symbol liệt kê tên hàm/biến mà file này **cung cấp** và **cần**.

Quan sát bằng `arm-none-eabi-nm main.o` (liệt kê symbol) hoặc `objdump -h` (liệt kê section).





- ▶ **Symbol resolution:** nối lời gọi `i2c_write` trong `main.o` tới định nghĩa thật trong `i2c_handler.o`.
- ▶ **Relocation:** điền địa chỉ tuyệt đối vào các "ô trống" – dựa trên bản đồ bộ nhớ do linker script quy định.
- ▶ **Garbage collection:** với `-Wl,--gc-sections`, hàm không ai gọi bị **loại khỏi** `.elf` → firmware gọn hơn.

Giả sử ta quên khai báo `uart_handler.c` trong `CMakeLists.txt`:

```
main.c:(.text+0x4a): undefined reference to `uart_log'
collect2: error: ld returned 1 exit status
```

## Đọc lỗi này thế nào

- ▶ *Compile* đã xong (mỗi `.o` tự dịch được).
- ▶ Lỗi ở **linker**: có lời gọi `uart_log` nhưng **không file `.o` nào cung cấp** định nghĩa.

## Cách sửa

Thêm file nguồn thiếu vào `target_sources(...)` hoặc liên kết đúng thư viện, rồi cấu hình lại.

Quy tắc nhớ: *undefined reference* = "thiếu file/thư viện" (tầng link); *expected ';' = "sai cú pháp"* (tầng compile).

| File | Nội dung                                      | Dùng để                             |
|------|-----------------------------------------------|-------------------------------------|
| .elf | mã máy + section + <b>symbol/debug</b>        | debug (GDB), phân tích, đo size     |
| .hex | dữ liệu + <b>địa chỉ</b> (Intel HEX, ASCII)   | nạp bằng nhiều công cụ khác nhau    |
| .bin | <b>chỉ</b> byte nhị phân thuần, không địa chỉ | nạp thẳng vào một địa chỉ cho trước |

## Vì sao có nhiều định dạng

.elf là bản “đầy đủ” nhưng có thông tin thừa với chip.  
.bin/.hex là bản “rút gọn” chỉ chứa thứ thật sự ghi vào flash.

## Lưu ý khi nạp .bin

.bin **không** mang địa chỉ, nên khi nạp phải **chỉ rõ** 0x08000000 cho công cụ (thấy ở Phần 5).

Trong CMakeLists.txt, sau khi có .elf, hai lệnh objcopy tự chạy (post-build):

```
arm-none-eabi-objcopy -O ihex    project.elf project.hex # trích ra Intel HEX
arm-none-eabi-objcopy -O binary  project.elf project.bin # trích ra binary thuần
arm-none-eabi-size              project.elf               # in kích thước text/data/bss
```

## Điều đang xảy ra

- ▶ objcopy “lột” phần debug/symbol khỏi .elf, chỉ giữ dữ liệu sẽ nằm trên chip.
- ▶ Nhờ vậy, một lần build cho ra đủ ba file mà **không cần gõ tay** – CMake lo phần này.

Trong thư mục build/ vừa tạo ở Check-in, chạy:

```
arm-none-eabi-size build/stm32f103_aht20_baremetal.elf
#  text    data    bss    dec    hex filename
#  ....    ...    ...    ....  ....  ...
```

## Câu hỏi thảo luận

- ▶ Cột text (mã + hằng số) lớn hơn hay data lớn hơn? Vì sao text chiếm FLASH?
- ▶ text + data chiếm bao nhiêu trong 64 KB FLASH? Còn dư bao nhiêu chỗ?
- ▶ data + bss chiếm bao nhiêu trong 20 KB RAM?
- ▶ Mở file .map và tìm dòng chứa Reset\_Handler – nó nằm ở địa chỉ nào?

**GIẢI LAO**

# PHẦN 2

## Bộ nhớ MCU & Linker script

Firmware nằm ở đâu trên chip?

**FLASH (rx)**  
**0x0800 0000**  
**– 0x0800 FFFF**  
**64 KB**  
`.isr_vector + .text + .rodata`

**RAM (xrw)**  
**0x2000 0000**  
**– 0x2000 4FFF**  
**20 KB**  
`.data + .bss + heap/stack`

## Hai vùng, hai vai trò

- ▶ **FLASH** (bền, đọc-thực-thi): giữ chương trình ngay cả khi mất điện → nơi firmware “định cư”.
- ▶ **RAM** (nhanh, đọc-ghi, mất khi tắt nguồn): nơi biến làm việc và ngăn xếp (stack) sống lúc chạy.

Con số này **không phải quy ước ngẫu nhiên** – nó do kiến trúc chip quy định và được ghi thẳng trong `STM32F103XX_FLASH.ld`.



Trích nguyên văn từ STM32F103XX\_FLASH.ld:

```
MEMORY
{
    RAM    (xrw) : ORIGIN = 0x20000000, LENGTH = 20K
    FLASH (rx)   : ORIGIN = 0x08000000, LENGTH = 64K
}
_estack = ORIGIN(RAM) + LENGTH(RAM); /* đỉnh RAM = day của stack */
```

## Ý nghĩa

- ▶ Khai báo cho linker biết **có những vùng nhớ nào**, bắt đầu ở đâu, dài bao nhiêu, quyền gì (r/w/x).
- ▶ \_estack đặt đỉnh ngăn xếp ở **cuối RAM** – stack lớn dần **đi xuống** từ đây.
- ▶ Nếu code/data vượt LENGTH, linker báo lỗi region overflowed – an toàn hơn tràn âm thầm.

```
SECTIONS
{
    .isr_vector : { KEEP(*(.isr_vector)) } >FLASH /* bang vector ngat, dat DAU tien */
    .text       : { *(.text*) *(.rodata*) } >FLASH /* ma lenh + hang so */
    ...
}
```

## Thứ tự quan trọng

- ▶ `.isr_vector` **bắt buộc** nằm ngay `0x08000000` – CPU tìm bảng vector đúng ở đó khi reset.
- ▶ `KEEP(...)` chặn `--gc-sections` xóa nhầm bảng vector (dù “không ai gọi”).

## >FLASH nghĩa là gì

Toán tử `>FLASH` bảo linker: “đặt section này vào vùng FLASH”. Đây là cách bản đồ bộ nhớ được áp lên firmware.

```
_sdata = LOADADDR(.data);          /* địa chỉ TAI (LMA) - nằm trong FLASH */  
.data : { _sdata = .; *(.data*) _edata = .; } >RAM AT> FLASH /* chạy ở RAM (VMA) */
```

## Vì sao .data có hai địa chỉ

- ▶ Biến có giá trị đầu (`int x = 5;`) phải **lưu bên** → giá trị nằm trong **FLASH** (LMA).
- ▶ Nhưng biến cần **ghi được** lúc chạy → phải sống trong **RAM** (VMA).

## Ai chép từ FLASH sang RAM?

Reset\_Handler (startup) chép khối .data từ \_sdata (FLASH) sang \_sdata..\_edata (RAM) **trước khi** vào `main()`.

AT> FLASH = “địa chỉ tài nằm ở FLASH”; >RAM = “địa chỉ chạy nằm ở RAM”. Hai khái niệm LMA/VMA là chìa khóa hiểu startup.

```
.bss : { _sbss = .; *(.bss*) *(COMMON) _ebss = .; } >RAM /* bien = 0 */  
_user_heap_stack : { . = . + 0x200; . = . + 0x400; } >RAM /* heap + stack toi thieu */
```

## .bss – “chỗ trống được đặt về 0”

Biến toàn cục không khởi tạo (`static int c;`) **không tốn FLASH** – linker chỉ cần biết **cần bao nhiêu RAM**; startup sẽ memset về 0.

## \_Min\_Stack\_Size

Dòng `. = . + 0x400` “giữ chỗ” 1KB cho stack. Nếu tổng vượt 20 KB RAM → lỗi lúc link, không phải khi chạy.

Nhờ tính trước ở tầng link, ta biết firmware có “vừa RAM” hay không **trước khi** nạp xuống board.

# Khi bấm reset, chuyện gì xảy ra?



- ▶ Hai từ đầu của bảng vector (.isr\_vector) chính là **giá trị stack pointer** và **địa chỉ Reset\_Handler**.
- ▶ Reset\_Handler dựng môi trường C (.data, .bss) rồi mới trao quyền cho main() – đó là lý do biến toàn cục có giá trị đúng ngay dòng đầu main.
- ▶ SystemInit() (trong system\_stm32f1xx.c) cấu hình PLL đưa clock lên 72 MHz trước khi ứng dụng chạy.



## Đọc được & sửa được

- ▶ Chuyển sang chip có 128 KB flash: chỉ đổi LENGTH.
- ▶ Đặt bootloader ở đầu flash: dời ORIGIN của ứng dụng.
- ▶ Đưa một hàm chạy trong RAM (tốc độ cao): dùng section .RamFunc.

## Gỡ lỗi vùng nhớ

- ▶ "region FLASH overflowed": firmware quá to.
- ▶ Chương trình chạy sai sau khi thêm mảng lớn: có thể **tràn stack** đè lên .bss.

Buổi học này chỉ cần **đọc hiểu vai trò** từng phần. Việc tự viết một linker script từ đầu sẽ để dành cho chủ đề bootloader về sau.

# PHẦN 3

## CMake – mô tả dự án một lần

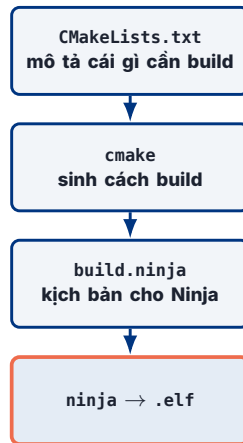
Một file mô tả, độc lập IDE

## Định nghĩa

CMake **không phải** trình biên dịch, cũng **không** trực tiếp build. Nó là **bộ sinh cấu hình**: đọc CMakeLists.txt rồi tạo file cho công cụ build thật sự (ở đây là Ninja).

## Không có CMake thì sao?

Bạn phải tự gõ tay hàng chục cờ gcc (include path, -mcpu, -D, đường dẫn .ld...) **mỗi lần** build – dễ sai, khó chia sẻ.





```
set(CMAKE_PROJECT_NAME stm32f103_aht20_baremetal)
include("cmake/gcc-arm-none-eabi.cmake") # 1. chỉ định cross-compiler
project(${CMAKE_PROJECT_NAME})
enable_language(C ASM) # baremetal: có cả file .s
add_executable(${CMAKE_PROJECT_NAME}) # 2. khai báo "sản phẩm" cần tạo

target_sources(${CMAKE_PROJECT_NAME} PRIVATE
    main.c i2c_handler.c uart_handler.c
    startup_stm32f103xb.s system_stm32f1xx.c) # 3. gom source
```

## Ba việc cốt lõi

- ❶ `include(...cmake)` nạp toolchain (build cho ARM, không phải PC).
- ❷ `add_executable` khai báo mục tiêu.
- ❸ `target_sources` liệt kê mọi file nguồn – **quên file nào ở đây** chính là nguồn của lỗi `undefined reference`.

```
target_compile_definitions(${CMAKE_PROJECT_NAME} PRIVATE
    STM32F103xB USE_FULL_ASSERT HSI_VALUE=8000000
    $<$<CONFIG:Debug>:DEBUG>)          # chỉ định nghĩa DEBUG khi build Debug

target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE
    ${CMAKE_SOURCE_DIR})              # nơi tìm file .h
```

## compile\_definitions = -D

Tương đương -DSTM32F103xB trên dòng lệnh gcc.  
STM32F103xB chọn đúng nhánh header cho chip;  
HSI\_VALUE khai báo tần số dao động nội.

## Biểu thức generator

`$<$<CONFIG:Debug>:DEBUG>` nghĩa là "chỉ thêm DEBUG khi cấu hình là Debug" – một macro, hai kiểu build.

```
add_custom_command(TARGET ${CMAKE_PROJECT_NAME} POST_BUILD
  COMMAND ${CMAKE_OBJCOPY} -O ihex  ${<TARGET_FILE:...>} project.hex
  COMMAND ${CMAKE_OBJCOPY} -O binary ${<TARGET_FILE:...>} project.bin
  COMMAND ${CMAKE_SIZE}                ${<TARGET_FILE:...>} # in size
```

## Vì sao gắn vào CMake

- ▶ Mỗi lần .elf được tạo, ba lệnh này **tự chạy** → luôn có .hex/.bin mới nhất, đồng bộ với code.
- ▶ ADDITIONAL\_CLEAN\_FILES khai báo .map/.hex/.bin để lệnh clean dọn sạch – tránh nạp nhầm file cũ.

Đây chính là lý do ở Check-in, chỉ một lệnh build mà có ngay đủ ba đầu ra.



Trích cmake/gcc-arm-none-eabi.cmake:

```
set(CMAKE_SYSTEM_NAME    Generic)          # không phải Windows/Linux -> baremetal
set(CMAKE_SYSTEM_PROCESSOR arm)
set(TOOLCHAIN_PREFIX      arm-none-eabi-)
set(CMAKE_C_COMPILER      ${TOOLCHAIN_PREFIX}gcc)
set(CMAKE_OBJCOPY         ${TOOLCHAIN_PREFIX}objcopy)
set(CMAKE_SIZE            ${TOOLCHAIN_PREFIX}size)
```

## Điểm cốt lõi

- ▶ CMAKE\_SYSTEM\_NAME Generic báo cho CMake: đích là hệ **không có HĐH** – không tự thêm thư viện của PC.
- ▶ Mọi công cụ (gcc, objcopy, size) đều mang tiền tố arm-none-eabi- thay vì bản của máy chủ.

```
set(TARGET_FLAGS "-mcpu=cortex-m3")      # dung loi CPU cua Blue Pill
... -ffunction-sections -fdata-sections   # tach section de bo code chet
... -T "STM32F103XX_FLASH.ld"            # nhung linker script
... --specs=nano.specs                   # newlib-nano: thu vien C gon
... -Wl,--gc-sections                    # linker X0A section khong dung
... -Wl,-Map=project.map -Wl,--print-memory-usage # xuat ban do + bao % bo nho
```

## Nhóm cờ CPU & linker

-mcpu=cortex-m3 sai → sinh lệnh chip không hiểu. -T ...ld nối linker script vào mọi lần link.

## Nhóm cờ tối ưu dung lượng

nano.specs + --gc-sections giúp firmware vừa 64 KB. --print-memory-usage in ngay % flash/RAM đã dùng.

Đổi sang chip khác (VD STM32F4) thường chỉ cần sửa **một file này** – CMakeLists.txt chính và code ứng dụng không đổi.



```
cmake -B build -G Ninja      # -B build: mọi file sinh ra đều vào thư mục build/
```

## Lợi ích

- ▶ Mã nguồn (.c/.h) **sạch**, không lẫn file trung gian .o/.ninja.
- ▶ Xóa sạch build chỉ cần xóa một thư mục build/.
- ▶ .gitignore chỉ cần bỏ qua build/ là đủ.

## Trong build/ có gì

build.ninja, các .o, .elf/.hex/.bin, .map, và compile\_commands.json (phục vụ clangd/VS Code index).

# PHẦN 4

## Ninja & tự động hóa bằng script

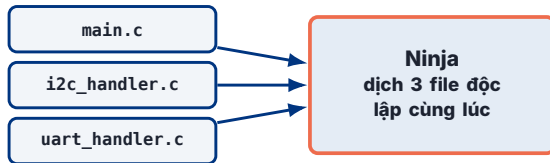
Build nhanh – và gói thành một lệnh

## Make truyền thống

- ▶ Đọc Makefile viết tay, tự dò phụ thuộc lúc chạy → chậm với dự án lớn.
- ▶ Cấu hình song song thủ công (-j).

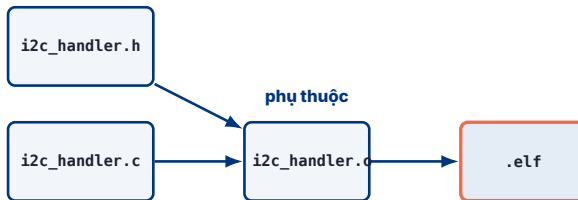
## Ninja

- ▶ build.ninja do CMake **sinh sẵn**, đã liệt kê rõ phụ thuộc → Ninja chỉ đọc & chạy.
- ▶ **Tự** song song hóa các file độc lập.



Ba file .c không phụ thuộc nhau nên có thể biên dịch đồng thời trên nhiều lõi CPU – rút ngắn thời gian build.





- ▶ Ninja lưu **dấu thời gian**: chỉ file nào (hoặc header nó phụ thuộc) thay đổi mới được dịch lại.
- ▶ Sửa `i2c_handler.c` → chỉ dịch lại `i2c_handler.o` rồi link – không đụng `main.o`, `uart_handler.o`.
- ▶ Sửa một `.h` dùng chung → **mọi** `.o` phụ thuộc nó được dịch lại. Đó là lý do đôi khi build “lâu bất thường”.

## Bước 1 – Cấu hình (sinh build.ninja):

```
cmake -B build -G Ninja
```

## Bước 2 – Biên dịch:

```
ninja -C build # tương đương: cmake --build build
```

## Khi nào chạy lại bước nào

- ▶ Bước 1 **chỉ** chạy lại khi đổi cấu trúc dự án: thêm file .c, sửa CMakeLists.txt, đổi cờ.
- ▶ Bước 2 chạy lại **mỗi lần sửa code** – Ninja tự biết file nào cần build lại.

Thử ngay tại lớp: sau khi đã build một lần, chạy lại `ninja -C build`:

```
$ ninja -C build
ninja: no work to do.          # không sửa gì -> không dịch lại gì
```

Sau đó sửa một dòng trong `main.c` rồi chạy lại:

```
$ ninja -C build
[1/2] Building C object .../main.o    # chỉ dịch lại main.o
[2/2] Linking C executable ...elf     # rồi link
```

## Bài học

Vòng lặp phát triển nhúng = **sửa code** → **build tăng dần** →  **nạp**. Build tăng dần khiến bước giữa gần như tức thì, giúp lặp nhanh.

# Vì sao gói mọi thứ vào một script?

## Gõ tay mỗi lần

```
rmdir /s /q build  
cmake -B build -G Ninja  
cmake --build build  
STM32_Programmer_CLI ...
```

→ 4 lệnh, dễ quên bước, dễ sai thứ tự

## Một script

build\_and\_flash.bat

→ một lệnh, luôn đúng thứ tự, ai chạy cũng ra kết quả như nhau

Script (.bat trên Windows, .sh trên Linux) chỉ đơn giản là **đóng gói một chuỗi lệnh terminal** vào một file, để gọi lại bằng một dòng duy nhất. Đây là bước đầu tiên của *tự động hóa có thể tái lập*.

```
REM [1] Don dep build cu
if exist build ( rmdir /s /q build )

REM [2] Cau hinh voi CMake
cmake -B build -G Ninja
if %ERRORLEVEL% neq 0 ( echo Configuration failed! & exit /b 1 )

REM [3] Bien dich
cmake --build build
if %ERRORLEVEL% neq 0 ( echo Build failed! & exit /b 1 )
```

## Ý chính

Logic “dọn dẹp → cấu hình → build” được viết **một lần**, lấy nguyên văn từ file thật của dự án.

```
cmake -B build -G Ninja  
if %ERRORLEVEL% neq 0 ( echo Configuration failed! & exit /b 1 )
```

## Vì sao phải dừng sớm

- ▶ Nếu cấu hình lỗi mà vẫn chạy tiếp → build trên nền hỏng, thông báo lỗi **gây hiểu nhầm**.
- ▶ `exit /b 1` dừng ngay, trả **mã lỗi khác 0**.

## Vì sao mã trả về quan trọng

Máy chủ CI/CD dựa vào **mã trả về** để biết “pass hay fail”. Một script trả 0 khi thất bại sẽ khiến CI báo xanh nhầm.

Trên Linux/.sh, tương đương là `set -e` hoặc kiểm tra `$?` sau mỗi lệnh.

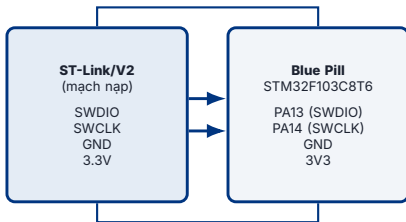
**GIẢI LAO**

# PHẦN 5

## Nạp firmware qua SWD & CLI

ST-Link, SWD & STM32\_Programmer\_CLI





## SWD (Serial Wire Debug)

- ▶ Chuẩn ARM dùng **2 dây tín hiệu** (SWDIO + SWCLK) cho **cả nạp lẫn debug**.
- ▶ Thêm GND chung và nguồn 3.3V → tổng 4 dây.

Không có SWD, máy tính không có “cửa” nào để chạm vào flash bên trong chip – không nạp được, cũng không debug được.

# ST-Link đóng vai trò gì trong chuỗi?



- ▶ PC nói **USB**; MCU chỉ hiểu **SWD**. ST-Link là **bộ chuyển ngữ** giữa hai bên.
- ▶ Phần mềm trên PC gửi lệnh “ghi khối byte này vào địa chỉ kia”; ST-Link thực thi qua SWD.
- ▶ Mỗi ST-Link có **số serial** riêng ( $s_n = \dots$ ) – hữu ích khi cắm nhiều board cùng lúc để chọn đúng thiết bị.

# “Flash” nghĩa là gì, chính xác?



- ▶ “Flash” = **ghi dữ liệu nhị phân** của .bin vào đúng địa chỉ gốc của bộ nhớ Flash vật lý.
- ▶ Với STM32, địa chỉ gốc luôn là **0x08000000** – do kiến trúc chip quy định, khớp với `ORIGIN(FLASH)` trong linker script.
- ▶ Sau khi ghi xong và **reset**, CPU đọc 4 byte đầu tại 0x08000000 làm stack pointer, rồi nhảy tới `Reset_Handler` – vòng đời khép lại với Phần 2.

```
STM32_Programmer_CLI -c port=SWD -w build/project.bin 0x08000000 -v -rst
```

| Tham số              | Ý nghĩa                                                 |
|----------------------|---------------------------------------------------------|
| -c port=SWD          | kết nối qua ST-Link, giao thức SWD                      |
| -w <file> 0x08000000 | <i>write</i> – ghi file vào địa chỉ Flash gốc           |
| -v                   | <i>verify</i> – đọc lại, so khớp dữ liệu vừa ghi        |
| -rst                 | <i>reset</i> – khởi động lại MCU, chạy chương trình mới |
| sn=...               | (tùy chọn) chọn đúng ST-Link theo số serial             |

Vì nạp file .bin (không mang địa chỉ), **bắt buộc** phải chỉ rõ 0x08000000 – sai địa chỉ này, chip sẽ không khởi động đúng.



```
REM ... clean, cmake -B build -G Ninja, cmake --build build (như Phan 4) ...
```

```
REM [4] Nạp firmware xuống board và reset
```

```
STM32_Programmer_CLI -c port=SWD ^  
-w build/project.bin 0x08000000 -v -rst
```



Một file .bat/.sh = **một lệnh** từ sửa code đến board chạy chương trình mới – viên gạch đầu tiên của pipeline CI/CD trong phát triển nhúng.

## GUI (STM32CubeProgrammer)

- ▶ Trực quan, tốt để khám phá lần đầu.
- ▶ Nhưng cần **người bấm chuột** – không tự động được.
- ▶ Không chạy trên máy chủ không màn hình.

## CLI (...\_Programmer\_CLI)

- ▶ Gọi được từ script, từ CI/CD, từ dây chuyền kiểm thử.
- ▶ Lặp lại **chính xác** như nhau mỗi lần.
- ▶ Ghi log, kiểm được mã trả về.

GUI dạy bạn *hiểu*; CLI để bạn *tự động hóa*. Một kỹ sư nhúng chuyên nghiệp dùng cả hai đúng lúc.

một script gọi trộn cả chuỗi



**Nhìn xa hơn:** thay "chạy script trên máy mình" bằng "máy chủ tự chạy script mỗi khi có code mới đẩy lên" – đó chính là **CI/CD** cho firmware. Đó là chủ đề của buổi tiếp theo.

- ▶ **Luồng biên dịch:** preprocess → compile → assemble → link, và ý nghĩa của .elf/.bin/.hex.
- ▶ **Bộ nhớ & linker script:** FLASH/RAM, các section, LMA vs VMA, và trình tự startup sau reset.
- ▶ **CMake:** mô tả dự án một lần, độc lập IDE; toolchain file cô lập "build cho chip nào".
- ▶ **Ninja & script:** build song song, tăng dần; đóng gói chuỗi lệnh thành một lệnh, kiểm mã trả về.
- ▶ **SWD + STM32\_Programmer\_CLI:** nạp .bin vào 0x08000000 rồi reset – toàn bộ bằng dòng lệnh.

## Bài tập về nhà

Tự viết build\_and\_flash.bat/.sh hoàn chỉnh cho i2c\_demo\_baremetal trên board của mình, đủ bốn bước: dọn dẹp – cấu hình – build – flash + reset, có kiểm tra mã lỗi ở mỗi bước.



| Thông báo / triệu chứng         | Tầng & hướng xử lý                            |
|---------------------------------|-----------------------------------------------|
| command not found               | môi trường: công cụ chưa trong PATH           |
| expected `;' before...          | compile: lỗi cú pháp C, sửa tại dòng báo      |
| undefined reference to ...      | link: thiếu file nguồn trong target_sources   |
| region FLASH overflowed         | link: firmware > 64 KB, bật -Os/--gc-sections |
| Error: No STLINK detected       | phần cứng: kiểm cáp USB / driver ST-Link      |
| Nạp xong nhưng board không chạy | sai địa chỉ -w ... 0x08000000 hoặc quên -rst  |

Quy tắc chung: đọc **dòng lỗi đầu tiên**, xác định **tầng** (môi trường / compile / link / nạp), rồi mới sửa.

# Thank You

---

## Acknowledgements

*Cảm ơn các bạn đã tham gia buổi học. Hẹn gặp lại ở buổi tiếp theo: Tự động hóa build & flash với CI/CD.*